

Dossier de Projet

Guardian Ledger

Application Full-Stack de gestion d'inventaire Destiny 2

Projet personnel — Formation Concepteur Développeur d'Applications

Dalstein Antoine

Remerciements

Je tiens à adresser mes sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet et à l'aboutissement de ma formation.

Je remercie en premier lieu **l'équipe pédagogique de l'école IT** pour la qualité de l'enseignement dispensé tout au long de la formation Concepteur Développeur d'Applications. Leurs retours, leur disponibilité et leur exigence m'ont permis de progresser régulièrement et d'aborder ce projet avec les bases solides nécessaires.

Je remercie également **l'entreprise Horloges-Huchez** et l'ensemble de mon équipe d'alternance pour m'avoir accordé leur confiance et pour m'avoir permis de mettre en pratique mes compétences en situation professionnelle réelle. Cette expérience en alternance a considérablement enrichi ma vision du développement logiciel et m'a appris à travailler dans un contexte d'équipe avec des contraintes concrètes.

Enfin, je remercie ma famille et mes proches pour leur soutien constant durant cette période de formation.

I. Introduction

Parcours

Je m'appelle **Dalstein Antoine**. Après l'obtention de mon **BTS SIO option SLAM** (Services Informatiques aux Organisations — Solutions Logicielles et Applications Métiers), j'ai souhaité approfondir mes compétences en développement logiciel en intégrant la formation **Concepteur Développeur d'Applications** à l'**École IT**, en alternance au sein de l'entreprise **Horloges-Huchez**.

Mon BTS SIO m'a apporté les fondamentaux du développement : algorithmique, bases de données relationnelles, développement web côté client et serveur, et premières notions de gestion de projet. La formation CDA m'a permis de franchir une nouvelle étape en me confrontant à des architectures plus complexes, à la conception orientée objet, aux bonnes pratiques de tests et à la démarche DevOps.

Contexte du projet

Guardian Ledger est né d'une double passion : le développement web et le jeu vidéo **Destiny 2**. L'objectif était de concevoir et développer, dans le cadre de ma formation CDA, une application full-stack complète et fonctionnelle permettant aux joueurs de Destiny 2 de consulter et gérer leur inventaire depuis un navigateur web, via l'API officielle de Bungie.net.

Le choix de ce sujet n'est pas anodin : en choisissant une API tierce complexe avec un protocole OAuth 2.0 complet, un manifeste de données massif (~300 Mo) et des contraintes techniques inattendues, je me suis volontairement placé dans des conditions proches d'un projet professionnel réel, bien au-delà d'un exercice académique classique.

Ce que ce projet m'a apporté

Ce projet est entièrement personnel. Il m'a permis de mettre en pratique l'ensemble des compétences acquises durant ma formation, en situation réelle de développement :

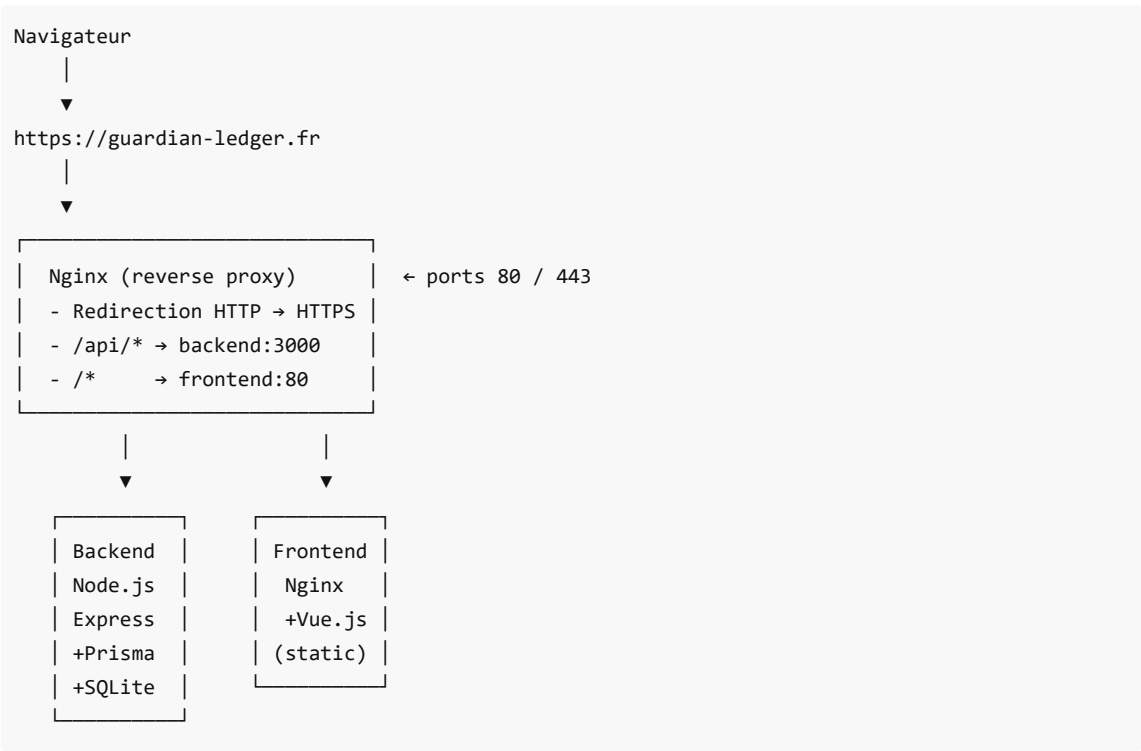
- **Conception** d'une architecture REST découplée frontend/backend
- **Authentification** double-couche (JWT applicatif + OAuth 2.0 Bungie)
- **Intégration d'API tierces** avec gestion du cycle de vie des tokens
- **Gestion de bases de données** avec Prisma ORM et SQLite
- **Tests automatisés** (66 tests, backend + frontend)
- **Conteneurisation** Docker et intégration continue GitHub Actions

Ce dossier de projet présente l'ensemble de cette réalisation, de la conception à la mise en place des tests, en passant par les choix techniques et les problèmes rencontrés.

II. Architecture technique

Vue d'ensemble

Guardian Ledger repose sur une architecture **découplée frontend/backend**, conteneurisée avec Docker et exposée via un reverse proxy Nginx. L'ensemble est déployé sur un VPS Linux accessible à l'adresse <https://guardian-ledger.fr>.



Stack technique

Couche	Technologie	Rôle
Frontend	Vue.js 3 + Vite	Interface utilisateur SPA
Routeur	Vue Router	Navigation côté client

Backend	Node.js + Express	API REST
ORM	Prisma	Accès base de données
Base de données	SQLite	Stockage utilisateurs et tokens
Manifeste	SQLite (~291 Mo)	Données statiques Destiny 2
Auth applicative	JWT (jsonwebtoken)	Sessions utilisateurs
Auth Bungie	OAuth 2.0	Accès à l'API Bungie.net
Reverse proxy	Nginx	Routage et HTTPS
SSL	Let's Encrypt / Certbot	Certificat TLS gratuit
Conteneurisation	Docker + Docker Compose	Isolation et déploiement
CI/CD	GitHub Actions	Tests automatisés + déploiement

Authentification double-couche

Le projet implémente deux niveaux d'authentification indépendants :

1. **JWT applicatif** — l'utilisateur crée un compte email/mot de passe sur Guardian Ledger. À la connexion, le backend génère un token JWT signé retourné au frontend et stocké en mémoire. Ce token est vérifié par le middleware `requireAuth` sur toutes les routes protégées.
2. **OAuth 2.0 Bungie** — pour accéder aux données Destiny 2, l'utilisateur autorise l'application via le portail Bungie.net. Le backend échange le code d'autorisation contre un `access_token` et un `refresh_token`, stockés en base de données et associés au compte utilisateur.

Cette architecture permet de dissocier l'identité applicative (compte Guardian Ledger) de l'identité Bungie, offrant ainsi une gestion fine des droits et une sécurité renforcée.

III. Modèle de données

Schéma Prisma

La base de données applicative (SQLite) contient une seule table `User` gérée via Prisma ORM :

```
model User {
  id          Int      @id @default(autoincrement())
  email       String   @unique
  password    String
  bungieMembershipId String? @unique
  bungieAccessToken String?
  bungieRefreshToken String?
  bungieTokenExpiresAt DateTime?
  displayName String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}
```

Ce modèle centralise les informations d'authentification applicative (`email` , `password` hashé avec `bcrypt`) et les tokens OAuth Bungie (`bungieAccessToken` , `bungieRefreshToken` , `bungieTokenExpiresAt`). La gestion de l'expiration du token permet au backend de détecter automatiquement les tokens périmés et de les renouveler via l'endpoint de refresh Bungie.

Manifeste Destiny 2

En parallèle, l'application utilise une base SQLite secondaire (`manifest.db` , ~291 Mo) contenant les définitions statiques du jeu Destiny 2 téléchargées depuis l'API Bungie. Ce fichier sert de dictionnaire local : à partir d'un hash numérique retourné par l'API (identifiant un objet, une arme, une armure...), le backend interroge le manifeste pour récupérer le nom, la description, l'icône et les statistiques associés.

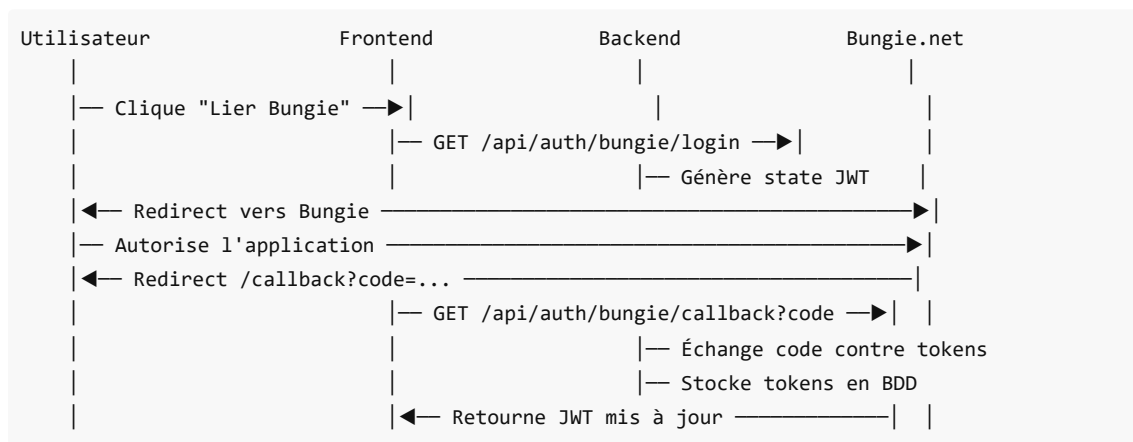
IV. Intégration de l'API Bungie

Contraintes spécifiques

L'API Bungie.net présente plusieurs particularités qui ont rendu son intégration particulièrement formatrice :

- **OAuth 2.0 complet** avec code d'autorisation, access token de courte durée et refresh token
- **Manifeste massif** (~300 Mo) à télécharger, décompresser et indexer dans SQLite
- **Hashes 32-bit signés** pour identifier les définitions d'objets, nécessitant une conversion spécifique (`hash >> 0`) pour correspondre aux IDs SQLite
- **Structure d'inventaire complexe** avec buckets (catégories d'emplacements) et instances d'objets

Flux OAuth



V. Fonctionnalités de l'application

Parcours utilisateur complet

Le parcours d'un utilisateur se déroule en quatre étapes distinctes :

1. **Création de compte** — l'utilisateur saisit un email et un mot de passe (min. 8 caractères). Le backend valide le format, hache le mot de passe avec `bcrypt` (coût 12) et crée l'entrée en base. Un `tempToken` JWT de courte durée (10 min) est retourné pour enchaîner immédiatement l'étape OAuth.
2. **Liaison du compte Bungie** — l'utilisateur est redirigé vers le portail Bungie.net pour autoriser l'application. Une fois l'autorisation accordée, le backend reçoit le code OAuth, l'échange contre un `access_token` et

un `refresh_token` , et lie le compte Bungie au compte Guardian Ledger. Un JWT applicatif final (validité 7 jours) est émis et stocké côté frontend.

3. **Dashboard** — vue principale après connexion. L'application interroge l'API Bungie pour récupérer le profil Destiny 2 complet de l'utilisateur et affiche :

- Les cartes de ses **gardiens** (personnages) avec classe, race et niveau de lumière
- Des **statistiques agrégées** : puissance maximale, puissance moyenne, nombre d'emplacements de coffre utilisés
- L'**équipement actuellement porté** par chaque gardien (casque, gantelets, torse, jambes, pièce de classe, armes)
- Possibilité de cliquer sur un item pour ouvrir la **fiche détaillée**

4. **Vault (coffre)** — vue dédiée à l'inventaire global. Affiche tous les items stockés dans le coffre du joueur, avec filtrage par recherche textuelle. Chaque item peut être transféré entre le coffre et un personnage via la fiche détaillée.

Fiche détaillée d'un item (`ItemDetailModal`)

En cliquant sur n'importe quel item, une modale affiche :

- L'icône, le nom, la rareté (commune, peu commune, rare, légendaire, exotique) et le type de l'item
- Le **niveau de puissance** (power light)
- La **description de lore** (`flavorText`) et la source d'obtention
- Les **statistiques** de l'item (portée, stabilité, maniabilité, rechargement, etc.)
- Les **perks** (avantages intrinsèques et sélectionnables) avec leurs icônes
- Une **barre de transfert** permettant d'envoyer l'item vers le coffre ou vers l'un des gardiens

Renouvellement automatique des tokens Bungie

L' `access_token` Bungie expire au bout d'une heure. Avant chaque appel à l'API Bungie, la fonction `ensureFreshToken()` vérifie la date d'expiration stockée en base. Si le token expire dans moins de 5 minutes, il est automatiquement renouvelé via le `refresh_token` sans interruption pour l'utilisateur.

Protection des routes (Navigation Guard)

Le routeur Vue Router intègre un `beforeEach` qui protège les routes `/dashboard` et `/vault` : si aucun token applicatif n'est présent dans le `localStorage` , l'utilisateur est redirigé vers `/login` . Inversement, un utilisateur déjà connecté accédant à `/login` est renvoyé directement vers son dashboard.

VI. Documentation de l'API REST

L'API backend expose trois groupes de routes, toutes préfixées par `/api` .

Routes d'authentification (`/api/auth`)

Méthode	Endpoint	Auth	Description
POST	<code>/api/auth/register</code>	Non	Crée un compte. Retourne un <code>tempToken</code> JWT (10 min)
POST	<code>/api/auth/login</code>	Non	Connexion. Retourne <code>appToken</code> (7j) ou <code>tempToken</code> si Bungie non lié

GET	/api/auth/bungie-connect	Non	Redirige vers le portail OAuth Bungie avec le state JWT
GET	/api/auth/callback	Non	Callback OAuth : échange le code, stocke les tokens, émet le JWT final

Exemple de réponse `POST /api/auth/login` (compte lié) :

```
{
  "appToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "displayName": "GuardianName#1234"
}
```

Routes utilisateur (/api/me)

Toutes les routes de ce groupe nécessitent un header `Authorization: Bearer <appToken>` .

Méthode	Endpoint	Description
GET	/api/me/destiny	Récupère le profil Destiny 2 complet (personnages, équipement, inventaire, instances)
POST	/api/me/transfer	Transfère un item entre un personnage et le coffre

Routes de données statiques (/api/data)

Méthode	Endpoint	Description
GET	/api/data/item/:hash	Retourne le nom, l'icône et le type d'un item à partir de son hash

Format des items retournés par /api/me/destiny

```
{
  "id": "6917529123456789",
  "itemHash": 2907129557,
  "name": "Gjallarhorn",
  "icon": "https://www.bungie.net/common/destiny2_content/icons/...",
  "power": 1810,
  "rarity": "exotic",
  "type": "Puissance",
  "bucketHash": 953998645,
  "guardianClass": "Universe1",
  "instanced": true
}
```

VII. Tests automatisés

Stratégie de test

Le projet compte **66 tests** répartis entre le backend (Jest) et le frontend (Vitest). L'objectif était de couvrir les comportements critiques de l'application : authentification, gestion des tokens, routes protégées et rendu des composants.

Tests backend (Jest)

Tous les tests backend utilisent des **mocks Jest** pour isoler les dépendances externes (Prisma, bcrypt, JWT, axios) et tester chaque unité de façon déterministe.

requireAuth.test.js — middleware JWT :

- Retourne 401 si le header `Authorization` est absent
- Retourne 401 si le header ne commence pas par `Bearer`
- Retourne 401 si le token est malformé ou expiré
- Attache `req.user` et appelle `next()` si le token est valide

routerAuth.test.js — routes d'authentification :

- `POST /register` : 400 si champs manquants, email invalide ou mot de passe trop court ; 409 si email déjà utilisé ; 200 avec `tempToken` si succès
- `POST /login` : 401 si utilisateur introuvable ou mot de passe incorrect ; retour `bungieRequired: true` + `tempToken` si Bungie non lié ; retour `appToken` si tout est lié
- `GET /callback` : 400 si state expiré ; échange du code OAuth et émission du JWT final

routerMe.helpers.test.js — logique de refresh token :

- Retourne le token existant s'il n'est pas encore expiré
- Déclenche un appel refresh si le token expire dans moins de 5 minutes
- Met à jour les tokens en base après un refresh réussi
- Lève une erreur si le refresh token est absent

bungieService.test.js — service Bungie :

- Envoie le header `X-API-Key` sur tous les appels
- Ajoute le header `Authorization: Bearer` si un token est fourni
- Retourne `response.data.Response` correctement extrait

manifestService.test.js — service manifeste :

- Convertit correctement un hash 32-bit non signé en entier signé (`hash >> 0`)
- Retourne l'objet JSON parsé depuis la ligne SQLite correspondante
- Retourne `null` si le hash est introuvable dans la table

Tests frontend (Vitest)

Les tests frontend utilisent `@vue/test-utils` avec Vitest pour monter les composants dans un environnement JSDOM.

index.test.js — routeur Vue Router :

- La route `/` redirige vers `/login`
- Les routes `/dashboard` et `/vault` ont le meta `requiresAuth: true`
- Le guard redirige vers `/login` si aucun token n'est stocké
- Le guard redirige vers `/dashboard` si un token est présent et que l'on accède à `/login`

HomeView.test.js — page d'accueil :

- Rendu sans erreur du composant
- Présence des éléments attendus (titre, CTA)

LoginView.test.js — formulaire de connexion/inscription :

- Affichage par défaut de l'onglet "Connexion"
- Bascule vers l'onglet "Créer un compte" au clic
- Affichage du message d'erreur si l'API retourne une erreur
- Appel à `POST /api/auth/login` avec les bonnes données à la soumission

Dashboard.test.js — tableau de bord gardiens :

- Affichage de l'overlay de chargement pendant la récupération des données
- Rendu des cartes gardiens après réception des données
- Affichage des statistiques (puissance max, moyenne, vault)

Vault.test.js — vue coffre :

- Chargement et affichage des items du coffre
- Filtrage par recherche textuelle

ItemDetailModal.test.js — modale de détail :

- Affichage du nom, de la rareté et du niveau de puissance de l'item
- Affichage de la barre de transfert si l'item est instancié
- Émission de l'événement `close` au clic sur le bouton de fermeture

VI. Pipeline CI/CD

Intégration continue (CI)

À chaque `push` ou `pull request` sur les branches `main` et `master`, GitHub Actions déclenche automatiquement deux jobs en parallèle :

- **Backend Tests** — installe les dépendances Node.js 22 et exécute les 66 tests Jest
- **Frontend Tests** — installe les dépendances et exécute les tests Vitest

Si l'un des jobs échoue, le merge est bloqué, garantissant qu'aucun code défaillant n'atteint la branche principale.

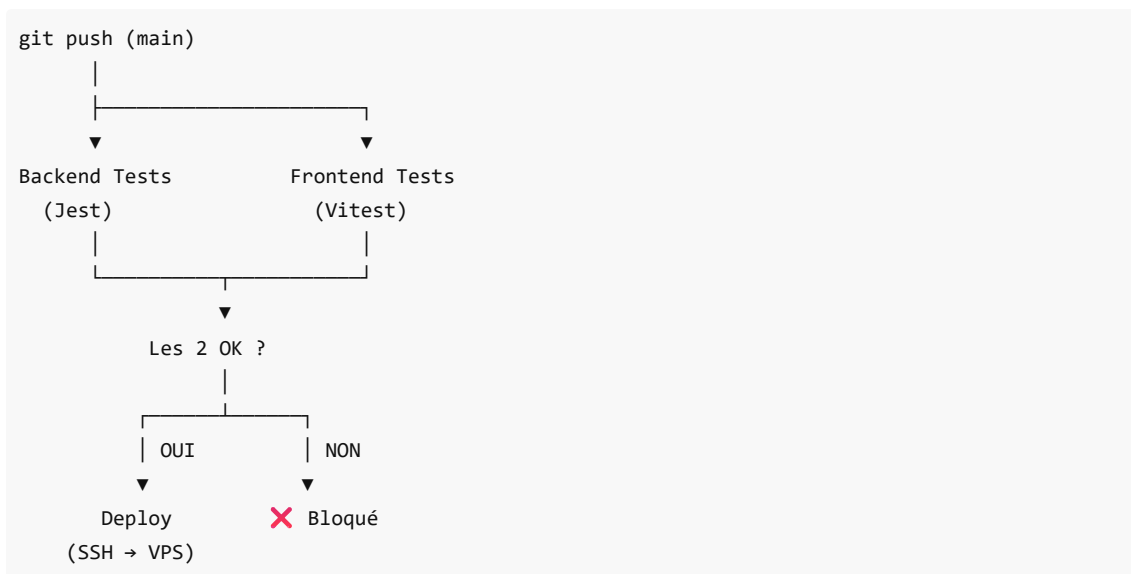
Déploiement continu (CD)

Un troisième job `deploy` se déclenche **uniquement après la réussite des deux jobs de tests** et **uniquement sur la branche principale**. Il se connecte au VPS via SSH et exécute :

```
cd ~/guardian_ledger
git pull
docker compose up -d --build backend frontend
docker compose restart nginx
```

Les secrets de connexion (`VPS_HOST`, `VPS_USER`, `VPS_SSH_KEY`) sont stockés dans les **GitHub Secrets** du repository et ne sont jamais exposés dans le code.

Schéma du pipeline



IX. Difficultés rencontrées et solutions

1. Les hashes 32-bit signés du manifeste Bungie

Problème : L'API Bungie retourne des identifiants numériques (hashes) pour tous les objets du jeu. Ces hashes sont des entiers 32-bit non signés (ex. 2907129557), mais SQLite les stocke en tant qu'entiers signés. Un hash supérieur à $2^{31} - 1$ devient négatif en base, ce qui rendait toutes les requêtes infructueuses.

Solution : Appliquer l'opération `hash >> 0` (décalage de bits de 0) en JavaScript, qui force la conversion en entier 32-bit signé, correspondant exactement à l'ID stocké par SQLite. Cette ligne unique dans `manifestService.js` a résolu l'intégralité des problèmes de lecture du manifeste.

```
const id = hash >> 0; // Conversion 32-bit non signé → signé
```

2. Sécurisation du flux OAuth avec le paramètre `state`

Problème : Le flux OAuth standard redirige l'utilisateur vers Bungie.net, qui rappelle ensuite le backend via `/callback`. Entre ces deux étapes, il est impossible de maintenir une session HTTP classique. Comment savoir quel utilisateur Guardian Ledger est en train d'effectuer la liaison Bungie ?

Solution : Utiliser le paramètre `state` du protocole OAuth comme vecteur d'information sécurisé. Lors du lancement du flux, le backend signe un JWT contenant le `userId` et le passe en `state`. Bungie retourne ce `state` intact dans le callback, permettant au backend de vérifier la signature JWT et de retrouver l'utilisateur sans exposition de données sensibles dans l'URL.

3. Volume du manifeste (~291 Mo)

Problème : Le manifeste Destiny 2 est un fichier ZIP de plusieurs centaines de mégaoctets. Le télécharger, le décompresser et l'indexer à chaque démarrage du serveur était impraticable.

Solution : Le manifeste est téléchargé une seule fois via le script `check_manifest.js` exécuté lors du build Docker. Le fichier `manifest.db` est persisté dans un volume Docker partagé, et `better-sqlite3` est utilisé pour des lectures synchrones très rapides sans connexion réseau. Le script vérifie la version du manifeste courante et ne re-télécharge que si Bungie publie une mise à jour.

4. Déduplication des items dans l'inventaire

Problème : L'API Bungie retourne les items de plusieurs sources (équipement actif, inventaire du personnage, coffre). Certains items non instanciés (matériaux, consommables) apparaissaient en double dans la réponse agrégée.

Solution : Implémentation d'une fonction `dedupe()` côté backend qui filtre les items sur leur `id` unique avant d'envoyer la réponse au frontend, garantissant qu'aucun item n'apparaît deux fois dans l'interface.

5. Isolation des dépendances dans les tests

Problème : Les tests backend dépendent de Prisma (accès BDD), de `bcrypt` (hash CPU-intensif), de `jsonwebtoken` et d' `axios` (appels réseau). Exécuter les vrais appels dans les tests les rendrait lents, non-déterministes et dépendants d'une base de données.

Solution : Utilisation des mocks Jest (`jest.mock()`) pour remplacer chaque dépendance par une implémentation contrôlée retournant des valeurs prédéfinies. Chaque test commence par `jest.clearAllMocks()` pour repartir d'un état propre et éviter les effets de bord entre tests.

X. Pipeline CI/CD — Détail de la configuration

Fichier de workflow GitHub Actions

Le fichier `.github/workflows/ci.yml` définit les trois jobs :

```
jobs:
  backend-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '22' }
      - run: npm ci
        working-directory: src/backend
      - run: npm test
        working-directory: src/backend

  frontend-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '22' }
      - run: npm ci
        working-directory: src/frontend
      - run: npm test
        working-directory: src/frontend

  deploy:
    needs: [backend-tests, frontend-tests]
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
```

```

- uses: webfactory/ssh-agent@v0.9.0
  with: { ssh-private-key: '${{ secrets.VPS_SSH_KEY }}' }
- run: |
    ssh '${{ secrets.VPS_USER }}@${{ secrets.VPS_HOST }} \
    "cd ~/guardian_ledger && git pull && \
    docker compose up -d --build backend frontend && \
    docker compose restart nginx"

```

Variables d'environnement

Les variables sensibles ne figurent jamais dans le code source. Elles sont injectées selon l'environnement :

Variable	Environnement	Usage
JWT_SECRET	Production (VPS)	Signature des tokens JWT
BUNGIE_API_KEY	Production + Dev local	Clé API Bungie
CLIENT_ID / CLIENT_SECRET	Production + Dev local	OAuth Bungie
VPS_HOST, VPS_USER, VPS_SSH_KEY	GitHub Secrets	Déploiement SSH

En développement local, ces variables sont définies dans un fichier `.env` non versionné (listé dans `.gitignore`).

XI. Bilan et perspectives

Ce que ce projet m'a apporté

Ce projet m'a permis de consolider et d'approfondir des compétences concrètes dans des domaines variés :

- **Conception d'API REST** : structuration des routes, gestion des codes HTTP, séparation des responsabilités entre middleware, routes et services
- **Sécurité applicative** : hachage des mots de passe, JWT, CSRF implicite via le paramètre `state` OAuth, HTTPS avec Let's Encrypt
- **Intégration de systèmes tiers** : gestion du cycle de vie OAuth 2.0, adaptation à une API non conventionnelle (hashes signés, manifeste massif)
- **Tests automatisés** : isolation par mocks, couverture des cas nominaux et d'erreur, tests de composants Vue
- **DevOps** : Dockerisation multi-service, reverse proxy Nginx, pipeline CI/CD GitHub Actions, déploiement automatisé sur VPS

Au-delà des compétences techniques, ce projet m'a appris à travailler avec des contraintes réelles : une API tierce dont le comportement peut changer, un manifeste de données massif, et des délais de développement à respecter dans le cadre de la formation.

Limites actuelles

- **Authentification frontend** : le token JWT est stocké dans le `localStorage`, ce qui l'expose à d'éventuelles attaques XSS. Une implémentation plus robuste utiliserait des cookies `HttpOnly`.
- **Fonctionnalités incomplètes** : plusieurs sections de l'interface (Collections, Triomphes, Carte, Season Pass) sont prévues dans la navigation mais non implémentées à ce stade.
- **Base SQLite** : SQLite est suffisant pour un usage personnel mais ne permettrait pas une mise à l'échelle vers de nombreux utilisateurs simultanés. Une migration vers PostgreSQL serait nécessaire pour un déploiement à grande échelle.

- **Couverture de tests frontend** : les tests des vues `Dashboard` et `Vault` restent superficiels. Une couverture plus complète nécessiterait de mocker les appels axios côté frontend.

Évolutions envisagées

- Implémentation des fonctionnalités de collections et triomphes
- Ajout d'un système de comparaison d'équipements
- Notifications push lors de l'expiration du token Bungie
- Migration vers des cookies `HttpOnly` pour sécuriser le stockage du JWT
- Tests end-to-end avec Playwright pour valider le parcours utilisateur complet

XII. Conclusion

Guardian Ledger est la concrétisation de ma formation CDA : une application full-stack complète, déployée en production, intégrant une API tierce complexe, une authentification double-couche, une suite de tests automatisés et un pipeline CI/CD opérationnel.

Ce projet m'a confronté à des problèmes réels qui ne figurent pas dans les exercices académiques : des hashes mal typés, un manifeste de 300 Mo à gérer, un flux OAuth sécurisé à construire de zéro, et des tokens à renouveler automatiquement. Chaque obstacle a été une opportunité d'apprendre à chercher, à comprendre et à résoudre — compétences que je considère aujourd'hui comme les plus précieuses acquises durant cette formation.

Le fait que l'application soit accessible en production à l'adresse <https://guardian-ledger.fr>, utilisable par n'importe quel joueur Destiny 2, est pour moi la meilleure validation de la démarche.

Glossaire

Terme	Définition
OAuth 2.0	Protocole d'autorisation permettant à une application d'accéder à des ressources d'un service tiers au nom d'un utilisateur, sans lui communiquer son mot de passe
JWT	JSON Web Token — token signé numériquement contenant des informations encodées, utilisé ici pour les sessions applicatives
Access Token	Token de courte durée (~1h) permettant d'effectuer des appels à l'API Bungie
Refresh Token	Token de longue durée permettant d'obtenir un nouvel access token sans redemander l'autorisation à l'utilisateur
Manifeste	Base de données SQLite (~291 Mo) contenant toutes les définitions statiques du jeu Destiny 2 (items, stats, perks...)
Hash Bungie	Identifiant numérique 32-bit utilisé par Bungie pour référencer tout objet de jeu
Bucket	Catégorie d'emplacement dans l'inventaire Destiny 2 (arme cinétique, casque, coffre...)
Instance	Exemplaire unique d'un item possédant ses propres statistiques (puissance, perks)
Prisma	ORM (Object-Relational Mapper) Node.js permettant d'interagir avec la base de données via des objets JavaScript typés

VPS	Virtual Private Server — serveur cloud Linux sur lequel l'application est déployée en production
SPA	Single Page Application — application web dont la navigation ne recharge pas la page entière